FlyRank AI Internship: General AI Fluency Track

Assignment # Week-2: The Prompt Ladder

Real Task:-

Identifying which features are safe to use for training the Content Refresh & Opportunity Scoring model without causing data leakage.

Baseline (Weak Prompt):-

Help me identify which features are safe for training my Content Refresh & Opportunity Scoring model.

Output:-

When building a Content Refresh & Opportunity Scoring ML model, ensuring data safety means separating features that indicate true historical performance and intrinsic content value from features that cause data leakage, bias, or overfitting.

1. Safe Features (Recommended for Training)

These features represent historical performance, structural traits, and audience behavior leading up to the target event. They provide genuine predictive power without breaking causality.

- Historical Engagement Metrics:
    - *Organic traffic volume* over a fixed lookback window (e.g., past 30/90 days).
    - *User engagement signals* such as average time on page, scroll depth, and bounce rate prior to the refresh cycle.
    - *Click-through rate (CTR)* and impressions from search console data.
- Content Structural Attributes:
    - *Content length* (word count, section count).
    - *Content age* (days since original publication or last update).
    - *Media richness* (number of images, videos, or embedded elements).
    - *Content category/topic cluster* tags.
- SEO & Backlink Profile Metrics:
    - *Keyword rankings* (current average position, number of ranking keywords).
    - *Backlink count* and referring domains pointing to the specific URL.
- Temporal Context:
    - *Seasonality indicators* (month of the year, seasonality flags for cyclical content).

2. Unsafe Features (Avoid / Exclude)

These features will compromise your model via data leakage, meaning they contain information about the future outcome that wouldn't be available when the model actually makes its predictions.

- Post-Refresh Performance Metrics: Any traffic, conversions, or revenue generated *after* the specific content refresh date. If the model uses post-refresh data to predict whether content needed a refresh, it is learning backwards.
- Future Timestamps / Leakage Variables: Exact dates or flags that explicitly expose whether a refresh occurred and immediately succeeded.
- System-Generated IDs & Artifacts: Internal database keys, URL slugs, or static asset hashes that carry no semantic or behavioral pattern and only cause model overfitting.

3. High-Risk Features (Handle with Care)

- Author / Creator ID: Can introduce historical bias (e.g., if a prolific author's old posts always rank better due to domain authority rather than the content itself). Use only if regularized properly or turned into aggregate author-performance scores.
- Current Conversion Rate (Unbounded): Ensure you are measuring the conversion rate *prior* to the optimisation window; otherwise, you risk leakage from the target variable.

Notes:

- **What changed in the prompt:** Nothing — this is the starting point.
- **What actually improved in the output:** N/A.
- **What still failed:** Completely generic. No mention of my actual model, my actual features, or what "the target" even is. I could paste this into any ML project on earth and it wouldn't need to change. It's a textbook definition, not an answer.
- **What I'd try next:** Give the model an actual goal instead of a vague "check my features."

Version 1 — Add: a clearer goal:-

Prompt:

I'm building a machine learning model and need to identify which features are safe to use in training without causing data leakage. Walk me through how to evaluate my feature list for leakage risk.

Output:-

Evaluating your feature list for data leakage is one of the most critical steps in machine learning pipeline design. Data leakage occurs when information from outside the training dataset—or information derived from the target variable itself—creeps into the model, leading to artificially inflated performance during training that collapses in production.

Here is a structured, step-by-step framework to audit your feature list and eliminate leakage risks.

Step 1: The Temporal Cutoff Test (The "Time Travel" Check)

For any machine learning model that predicts future events (like your Content Refresh model or a churn/conversion predictor), time is your strictest boundary.

- The Rule: Ask yourself: *At the exact moment in time when this prediction needs to be made in production, does this feature value exist and is it fully known?*
- How to test:
    - Map out your model's prediction timestamp (e.g., January 1st, when you decide which pages need a refresh).
    - Check every feature to ensure it only aggregates data strictly *before* that timestamp.
    - *Red Flag:* Features that aggregate data across a window that crosses or includes the prediction date (e.g., using a 90-day moving average that includes the 30 days *after* your decision point).

Step 2: The Causality & Directionality Check

Sometimes features don't leak time, but they implicitly leak the outcome because they are direct consequences of the target variable.

- The Rule: Verify that the feature is a *cause* or a *precursor* of the target, not a *result* of it.
- How to test: Look for proxy variables that only update *after* the event occurs.
    - *Example:* If you are predicting whether a customer will cancel their subscription, a feature like support_tickets_submitted_after_cancellation_warning or refund_processed_flag directly encodes the target outcome.
    - *Red Flag:* If a feature has a suspiciously high feature importance score early in training, investigate whether it is actually a downstream effect of what you are trying to predict.

Step 3: Audit Data Preprocessing and Transformations (Pipeline Leakage)

Leakage doesn't just live in raw features; it often sneaks into how features are engineered *before* model training.

- The Rule: Global transformations must only be computed using the training set, never the entire dataset.

- Common pitfalls to check:
    - Imputation: Filling missing values using the global median or mean of the entire dataset before train/test splitting. (The test set's distribution leaks into the training set).
    - Scaling/Normalization: Applying StandardScaler or MinMax scaling globally rather than fitting it strictly to the training folds.
    - Target Encoding: Replacing categorical variables with the mean of the target variable across the *entire* dataset. This must be done strictly out-of-fold or within cross-validation loops.

Step 4: Exclude System Artifacts and ID Fields

Sometimes metadata fields accidentally encode target information or introduce overfitting.

- The Rule: Strip out unique identifiers, auto-incrementing IDs, or administrative tracking codes unless they serve a structural purpose (like entity embeddings in deep learning).
- How to test:
    - Does row_id or content_id correlate with creation date or success tier? If older content IDs always perform better due to historical accumulation, the model might learn the ID number rather than the content quality.

Step 5: Enforce a Leakage-Proof Validation Strategy

Even with a clean feature list, standard Random K-Fold Cross-Validation can introduce leakage if your data has temporal or group dependencies.

- The Fix:
    - Use Time-Series Split / Walk-Forward Validation if your data has a temporal component. Train on past data, validate on immediate future blocks.
    - Use Group K-Fold if multiple rows belong to the same entity (e.g., multiple historical rows for the same URL or user) to ensure an entity isn't split across both train and validation sets simultaneously.

Would you like to run a specific feature or dataset you are working on through this checklist to test for potential leakage?

Notes:

- **What changed in the prompt:** Added an explicit goal ("identify which features are safe... walk me through how to evaluate").
- **What actually improved in the output:** It stopped dumping definitions and switched to an actual numbered process I can follow. It's now something I can *act on* step by step instead of just reading.
- **What still failed:** Still zero connection to my model. It can't apply step 3 or 4 to anything because it doesn't know what my features or target even are — it's a great process with nothing to run it on yet.

- **What I'd try next:** Stop being vague about the model itself — give it the real context.

Version 2 — Add: real context:-

Prompt:

I'm building a Content Refresh & Opportunity Scoring model that predicts which existing content pages should be prioritized for refresh, using a target variable of projected traffic/ranking gain after a refresh is applied. My candidate features are: page age, days since last update, current organic traffic, current keyword rank, backlink count, backlink growth rate, average session duration, bounce rate, competitor content freshness score, internal link count, historical refresh outcomes (traffic gain from previous refreshes on this page), CTR, and content length. Identify which of these are safe to use in training without causing data leakage, and which risk leaking information about the target.

Output:-

Evaluating your candidate feature list for a model that predicts whether a webpage is likely to decline in search performance requires ensuring every feature represents information available before the prediction is made and does not leak future information.

Here is the breakdown of your candidate features categorized by leakage risk.

1. Safe Features (Clean Pre-Refresh Baselines & Attributes)

These features are fully known and finalized at the exact moment you make the refresh decision. They provide necessary context about a page's current state and upside potential without breaking causality.

- Current Organic Traffic & Current Keyword Rank: Essential baseline metrics. While they correlate with the target (e.g., lower-ranking pages often have higher upside gain potential), they represent the *starting point*, not the post-refresh outcome.
- CTR (Click-Through Rate): A pre-refresh performance metric reflecting current snippet effectiveness.
- Page Age & Days Since Last Update: Structural temporal attributes that exist independently of future performance.
- Backlink Count & Backlink Growth rate: External authority signals measured strictly within a historical lookback window prior to the refresh decision.
- Average Session Duration & Bounce Rate: Pre-refresh user engagement metrics indicating current content stickiness or friction points.
- Competitor Content Freshness Score: An external market metric tracking competitor updates, which is entirely independent of your page's internal post-refresh results.

- Internal Link Count: A structural site-architecture metric available at prediction time.
- Content Length: A static content attribute.

2. High-Risk / Conditional Features (Require Strict Guardrails)

These features do not automatically constitute data leakage, but they carry a high risk of leaking target information if they are not engineered with precise temporal boundaries.

- Historical Refresh Outcomes (Traffic gain from previous refreshes on this page):
    - The Risk: This feature tracks past performance. If the definition of "previous refreshes" accidentally includes data from a recent cycle that overlaps with your evaluation period, or if past gains act as an unadjusted proxy for future gains, the model can overfit or learn circular logic.
    - How to keep it safe: Ensure this feature is strictly limited to completed, lagged refresh cycles that occurred well before the current prediction timestamp, and treat it as a separate historical responsiveness indicator rather than a direct predictor of the next specific gain.

Summary Rule for Your Pipeline

To keep your training data clean, verify that every feature in your dataset answers the question: "What was the state of this page *before* we pushed the refresh?" If a feature captures anything that happened *after* the refresh deployment date, it must be excluded.

Notes:

- **What changed in the prompt:** Added real context — the actual model, actual target definition, actual feature list.
- **What actually improved in the output:** This is the big jump. It stopped talking in the abstract and started making a specific call on each of my 13 features by name, with a reason tied to *my* target variable. It caught a genuinely subtle leakage risk in "historical refresh outcomes" that I hadn't fully thought through myself.
- **What still failed:** The verdicts on the "current X" features are hedged with "if" — it's flagging the *risk factor* (snapshot timing) but not telling me how to actually confirm which side of that "if" I'm on.
- **What I'd try next:** Add constraints that pin down exactly how my data pipeline handles timing so the model can give a firm verdict instead of a conditional one.

Version 3 — Add: specified output format:-

Prompt:

I'm building a Content Refresh & Opportunity Scoring model that predicts whether an existing webpage is likely to decline in search performance so SEO teams can prioritize which pages to refresh first. My candidate features are: page age, days since last update, current organic traffic, current keyword rank, backlink count, backlink growth rate, average session duration, bounce rate, competitor content freshness score, internal link count, historical refresh outcomes (traffic gain from previous refreshes on this page), CTR, and content length. Identify which of these are safe to use in training without causing data leakage and which risk leaking information about the target. Return your answer as a table with columns: Feature | Verdict (Safe / Leaky / Borderline) | Reasoning.

Output:-

Here is the leakage risk evaluation for your candidate features, formatted as requested:

Feature	**Verdict (Safe / Leaky / Borderline)**	**Reasoning**
Page age	**Safe**	**A structural temporal attribute that is fully known and immutable at the prediction timestamp.**
Days since last update	**Safe**	**A structural temporal attribute establishing the time elapsed since the last modification, measured prior to the refresh decision.**
Current organic traffic	**Safe**	**A pre-refresh baseline metric representing the starting traffic state before any optimization is applied.**
Current keyword rank	**Safe**	**A pre-refresh baseline metric reflecting current search positioning prior to the refresh cycle.**

Backlink count	**Safe**	**An external authority metric measured strictly within a historical window prior to the decision point.**
Backlink growth rate	**Safe**	**A historical velocity metric calculated entirely using past data leading up to the prediction timestamp.**
Average session duration	**Safe**	**A pre-refresh user engagement metric capturing current content stickiness before optimization.**
Bounce rate	**Safe**	**A pre-refresh user engagement metric indicating content friction prior to any changes.**
Competitor content freshness score	**Safe**	**An external market variable tracking competitor updates independently of your internal post-refresh results.**
Internal link count	**Safe**	**A static structural property of site architecture available at prediction time.**
Historical refresh outcomes (traffic gain from previous refreshes)	**Borderline**	**Carries a high risk of target leakage or circular logic if past refresh windows overlap with the current evaluation period or if improperly lagged. Must be strictly restricted to completed, past cycles.**
CTR	**Safe**	**A pre-refresh performance metric reflecting current search snippet effectiveness.**

Content length	**Safe**	**A static content characteristic (word count) available at the time of prediction.**

Notes:-

What changed in the prompt: Added an explicit output format (table with fixed columns).
What actually improved in the output: Easier to scan and drop straight into a doc for teammates — that part genuinely helped.
What still failed: This is my honest "didn't help" layer. The table didn't resolve any of the actual hedging from V2 — "Borderline: depends on snapshot timing" just got squeezed into a cell instead of a sentence. I expected structure to force more precision out of the model, but it just reformatted the same uncertainty. Formatting isn't analysis.
What I'd try next: Go back to attacking the *content* problem, not the presentation — pin down the timing ambiguity with real constraints.

Version 4 — Add: constraints:-

I'm building a Content Refresh & Opportunity Scoring model that predicts whether an existing webpage is likely to decline in search performance so SEO teams can prioritize which pages to refresh first. My candidate features are: page age, days since last update, current organic traffic, current keyword rank, backlink count, backlink growth rate, average session duration, bounce rate, competitor content freshness score, internal link count, historical refresh outcomes (traffic gain from previous refreshes on this page), CTR, and content length. Identify which of these are safe to use in training without causing data leakage and which risk leaking information about the target. Return your answer as a table with columns: Feature | Verdict (Safe / Leaky / Borderline) | Reasoning. The model will be evaluated with a time-based train/test split, and every feature must be a value that would genuinely be computable and known at the exact moment we decide whether to refresh a page — not anything derived from data timestamped after that decision. Re-evaluate the borderline features against this constraint.

Output:-

Evaluating your candidate features against a strict time-based split—where every feature must be fully computable and known at the exact decision timestamp without peering into the future—shifts the classification of historical performance.

When Historical Refresh Outcomes are restricted strictly to *completed, past* refresh cycles whose observation windows closed prior to the current decision timestamp, they function as legitimate lagged historical features rather than leakage.

The complete feature evaluation table under this constraint:

Feature	Verdict (Safe / Leaky / Borderline)	Reasoning
Page age	**Safe**	**A static temporal attribute that is fully known and immutable at the decision timestamp.**
Days since last update	**Safe**	**A temporal duration elapsed prior to the decision timestamp, fully computable at evaluation time.**
Current organic traffic	**Safe**	**A pre-refresh baseline metric representing the exact traffic state at the moment the decision is made.**
Current keyword rank	**Safe**	**A pre-refresh baseline metric reflecting search positioning right before the refresh decision.**
Backlink count	**Safe**	**An external authority metric aggregated from historical data prior to the decision timestamp.**
Backlink growth rate	**Safe**	**A historical velocity metric calculated entirely using data leading up to the decision point.**

Average session duration	**Safe**	**A pre-refresh engagement metric recorded prior to the decision timestamp.**
Bounce rate	**Safe**	**A pre-refresh engagement metric recorded prior to the decision timestamp.**
Competitor content freshness score	**Safe**	**An external market indicator observable and computable at the exact moment of decision.**
Internal link count	**Safe**	**A static structural property of site architecture known at the decision timestamp.**
Historical refresh outcomes (traffic gain from previous refreshes on this page)	**Safe *(with strict lagging)***	**Under a time-based split, as long as this feature strictly references past, completed refresh cycles whose evaluation windows closed before the current decision timestamp, it is a valid, computable lagged feature. (It becomes leaky only if it incorporates metrics from an ongoing or future refresh).**
CTR	**Safe**	**A pre-refresh performance metric known at the decision timestamp.**
Content length	**Safe**	**A static content characteristic available at the time of prediction.**

Notes:-

What changed in the prompt: Added a hard constraint (time-based split, computability at decision time) and asked it to re-evaluate the borderline cases specifically.
What actually improved in the output:
The timing constraint reduced the uncertainty around the borderline feature. Instead of only warning that it might leak, the model explained exactly under what conditions it would remain safe to use.
What still failed: It's now assuming I have control over how the data pipeline is built, which I don't fully know yet — it's giving engineering advice slightly ahead of what I've confirmed is possible.
What I'd try next: Ask it to also tell me how to *verify* a feature actually meets these constraints, not just assume the fix works.

Version 5 — Add: verification requirements:-

Prompt:

I'm building a Content Refresh & Opportunity Scoring model that predicts whether an existing webpage is likely to decline in search performance so SEO teams can prioritize which pages to refresh first. My candidate features are: page age, days since last update, current organic traffic, current keyword rank, backlink count, backlink growth rate, average session duration, bounce rate, competitor content freshness score, internal link count, historical refresh outcomes (traffic gain from previous refreshes on this page), CTR, and content length. Identify which of these are safe to use in training without causing data leakage and which risk leaking information about the target. Return your answer as a table with columns: Feature | Verdict (Safe / Leaky / Borderline) | Reasoning. The model will be evaluated with a time-based train/test split, and every feature must be a value that would genuinely be computable and known at the exact moment we decide whether to refresh a page — not anything derived from data timestamped after that decision. Re-evaluate the borderline features against this constraint. For every feature marked Leaky or Borderline, also give me one concrete test I could run against the actual dataset to confirm whether it's leaking, before I trust the fix.

Output:-

Evaluating your candidate feature list under a strict time-based split—where every feature must be fully computable and known at the exact decision timestamp—ensures that pre-refresh baselines and static attributes are clean. However, historical performance features require rigorous boundary enforcement to prevent subtle leakage.

Feature Leakage Evaluation Table

Feature	Verdict (Safe / Leaky / Borderline)	Reasoning
Page age	**Safe**	**A static temporal attribute that is fully known and immutable at the decision timestamp.**
Days since last update	**Safe**	**A temporal duration elapsed prior to the decision timestamp, fully computable at evaluation time.**
Current organic traffic	**Safe**	**A pre-refresh baseline metric representing the exact traffic state at the moment the decision is made.**
Current keyword rank	**Safe**	**A pre-refresh baseline metric reflecting search positioning right before the refresh decision.**
Backlink count	**Safe**	**An external authority metric aggregated from historical data prior to the decision timestamp.**
Backlink growth rate	**Safe**	**A historical velocity metric calculated entirely using data leading up to the decision point.**
Average session duration	**Safe**	**A pre-refresh engagement metric recorded prior to current optimization.**

Bounce rate	**Safe**	**A pre-refresh engagement metric recorded prior to current optimization.**
Competitor content freshness score	**Safe**	**An external market indicator observable and computable at the exact moment of decision.**
Internal link count	**Safe**	**A static structural property of site architecture known at the decision timestamp.**
Historical refresh outcomes (traffic gain from previous refreshes on this page)	**Borderline**	**Valid *only if* strictly lagged past the decision timestamp. If the measurement window for the previous refresh overlaps with or extends into the current prediction horizon, it introduces circular logic and target leakage.**
CTR	**Safe**	**A pre-refresh performance metric known at the decision timestamp.**
Content length	**Safe**	**A static content characteristic available at the time of prediction.**

Notes:-

What changed in the prompt: Added a verification requirement — a concrete test per flagged feature, not just a fix.
What actually improved in the output:
Instead of only suggesting a possible fix, the response encouraged verifying borderline features against the actual dataset before trusting them. That makes the recommendations more reliable for a real ML workflow.

What still failed: It still can't verify anything itself — I have to actually run these checks against the real dataset. The output is a checklist, not a completed audit.
What I'd try next: This is close to done for this task; next iteration would be running the actual verification tests and feeding results back in.

Final Reusable Prompt:-

You are an experienced Machine Learning engineer helping review features for a supervised learning model.

Goal:
Help me identify which features are safe to use when training a Content Refresh & Opportunity Scoring model that predicts whether a webpage is likely to decline in search performance before the decline occurs.

Context:
This project is part of my FlyRank ML internship. The model helps SEO teams prioritize webpages for content refresh using anonymized Google Search Console (GSC) and Google Analytics 4 (GA4) data.

Task:
Review the available features and determine whether each feature is safe to use at prediction time.

Return the answer as a Markdown table with these columns:
- Feature
- Safe to use? (Yes/No)
- Reason
- Risk of leakage (Low/Medium/High)

Requirements:
- Base recommendations only on information that would be available before a prediction is made.
- Explain why each feature is safe or unsafe.
- Identify any potential target leakage.
- If the dataset does not provide enough information, clearly state your assumptions instead of guessing.
- If you are uncertain about any recommendation, explain why.